

深圳大学实验报告

课程名称： 微机原理及应用

实验项目名称： 实验三 基于 PC 的简易整数四则运算器

学院： 物理与光电工程学院

专业： 物理学

指导教师： _____

报告人： Kuri すずは 学号： _____ 班级： _____

实验时间： 2025 年 11 月 18 日-2025 年 12 月 22 日

实验报告提交时间： 2025 年 12 月 23 日

教务处制

一、实验名称： 基于 PC 的简易整数四则运算器
二、实验目的 1.熟练掌握 8086 汇编语言编译调试工具和环境的操作； 2.掌握完整 8086 汇编的程序设计编写方法； 3.学会规范的程序流程图画法； 4.掌握简单的数值计算指令的应用方法； 5.掌握 DOS 功能调用方法； 6.学习与掌握 8086 汇编语言的分支程序、循环程序、子程序等设计方法。
三、实验内容 基于实验一和实验二，本实验将设计一个基于 PC 的简易整数四则运算计算器。键盘输入计算数值，程序计算结果，在屏幕输出结果。
(5-10%) 四、实验任务分析与算法分析（软件实验）或实验电路及连线（硬件实验） 1.任务分析： 本实验的核心任务是设计并实现一个基于 PC 的简易整数四则运算器，支持四位数（正整数）以内的加法、减法、乘法和除法运算。以下是三个主要任务： (1) 键盘输入部分：获取用户键入的两个四位以内的正整数和一个运算符； (2) 四则运算部分：根据键入的运算符执行加法、减法、乘法或除法运算； (3) 结果显示部分：将运算结果在屏幕上以适当的格式显示出来。 实验采用循序渐进的开发策略，在完成以上三个主要任务的基础上完善细节、做进一步优化，以逐步完善程序功能。以下是基于程序基本功能提出的增强功能： (1) 人机交互：能够在屏幕上显示输入提示语句和错误提示语句，并且能够读取用户键入的字符； (2) 模块化：对于代码重复度高、需要多次用到的部分，合理地设计多个子程序，使程序结构更加清晰，减少代码量； (3) 输入有效性判断：确保用户输入的数字在指定范围内（0001~9999），并且运算符合法（+、-、*、/）。对于非法输入，提示错误并要求重新输入； (4) 数据转换：根据运算和显示的需要，实现 ASCII 字符与数值之间的转换以及非压缩型 BCD 码与二进制数之间的转换； (5) 结果显示格式：加法和减法以竖式格式显示计算结果，需要实现每一位的对齐。乘法和除法以横式格式显示计算结果，若除法余数不为 0，需要显示省略号和余数； (6) 结果报错：减法和乘法运算可能会出现结果超出运算范围的情况，从而导致结果错误。为避免为用户提供错误答案，当运算结果错误时，在屏幕上提示结果错误； (7) 反复运算：允许用户多次进行运算，直到选择退出程序； (8) 去除前导零：最高位是 0 不显示，除非结果是 0。 因此，编程的大体思路是：先完成三个主要任务，构建好程序的主要框架。对不同分支进行调试并确定无误后，再增加程序的功能。最后做结构上的优化，并检查程序的细节错误。

2.算法分析:

(1) 输入处理算法

- 输入提示: 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 分别显示输入数和运算符的提示语句 (MSG1-MSG3);
- 输入方式: 输入数时, 循环调用 DOS 的 INT 21H 中断的 1 号功能 (MOV AH,1), 读取用户键入的 4 个字符。输入运算符时, 调用 DOS 的 INT 21H 中断的 1 号功能 (MOV AH,1), 读取用户键入的单个字符;
- 有效性判断: 输入数时, 用 CMP 语句对每个输入的字符进行判断, 确保其在'0'到'9'的范围内, 且除数不为 0, 否则提示错误并要求重新输入。输入运算符时, 用 CMP 语句对输入的字符进行判断, 确保其为'+','-','*','/'四种运算符之一。对于非法输入, 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9) 显示输入错误的提示语句 (MSG4-MSG5);
- 码制转换: 将有效的 ASCII 数字字符通过 SUB 30H 指令转换为对应的二进制数值。

(2) 运算处理算法

用 CMP 指令, 分别将输入的运算符 OPR 与'+','-','*','/'进行比较, 从而确定将进行以下四种运算的哪一种运算, 该部分采取分支结构。

• 加法运算

- ①初始化: 循环次数 CX=4, SI、DI、BX 分别指向被加数、加数、结果的个位, 采用 CLC 指令确保 CF=0, DL 用于存放最高位进位;
- ②加法循环: 取出被加数当前位, 使用 ADC 指令与加数当前位相加, 执行 AAA 指令进行非组合 BCD 数的十进制调整, 存入结果。三个指针全部向高位移动。LOOP 指令循环, 直到处理完每一位;
- ③处理最高位进位: 将 DL 中的数 (0/1) 存入结果的最高位, 运算完毕。

• 减法运算

- ①初始化: 循环次数 CX=5 (多 1 用于处理借位, 设为 4 会使下一次减法运算的最高位错误), SI、DI、BX 分别指向被减数、减数、结果的个位, 采用 CLC 指令确保 CF=0;
- ②减法循环: 取出被减数当前位, 使用 SBB 指令与加数当前位相加, 执行 AAS 指令进行非组合 BCD 数的十进制调整, 存入结果。三个指针全部向高位移动。LOOP 指令循环, 直到处理完每一位, 运算完毕。

• **乘法运算**: 将两个数由非组合 BCD 数转换成二进制数, 使用 MUL 指令实现无符号乘法。执行 MUL 后, 用 CMP 指令检查 DX 是否为 0, 若 DX 为 0, 说明得数在 0FFFFH 以内, 没有因超出范围而造成结果错误, 将错误标志 ERRORFLAG 置 1, 否则置 0。

• **除法运算**: 将两个数由非组合 BCD 数转换成二进制数, 使用 DIV 指令实现无符号除法。

(3) 结果处理算法

- 将积、商和余数由二进制数转换成非组合 BCD 数;
- 前导零去除: 通过 REMOVE_LEADING_ZEROS 子程序, 使用循环和 CMP 指令检测并替换前导零为空格。对于五位数的结果, 只需要循环检查前四位, 一旦检查到非'0'就退出循环。

(4) 结果显示算法

由于加减法和乘除法显示结果的格式不同, 对其运算结果的处理方法也有所不同。

- 对于**加减法**, 以下是结果显示的步骤:
 - ①调用子程序 SPACE 显示空格;
 - ②将被加数/被减数转换成 ASCII 字符, 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 显示被加数/被减数;
 - ③调用子程序 CRLF 显示回车换行;

- ④调用 DOS 的 INT 21H 中断的 2 号功能 (MOV AH,2), 显示单个字符 '+'/'-';
- ⑤将加数/减数转换成 ASCII 字符, 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 显示加数/减数;
- ⑥调用子程序 CRLF 显示回车换行;
- ⑦调用子程序 SEPARATOR 显示分割线;
- ⑧调用子程序 CRLF 显示回车换行;
- ⑨调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 显示和/差;
- ⑩ (此步骤用于判断减法的结果错误显示) 用 CMP 指令比较被减数和减数, 若被减数大于减数, 计算结果将为负数, 结果将会显示错误, 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 显示错误提示语句 MSG6。

通过以上步骤, 可以实现加法和减法的竖式计算, 各位都是一一对齐的。

• 对于乘法, 以下是结果显示的步骤:

- ①将被乘数转换成 ASCII 字符, 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 显示被乘数;
- ②调用 DOS 的 INT 21H 中断的 2 号功能 (MOV AH,2), 显示运算符 '*';
- ③将乘数转换成 ASCII 字符, 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 显示乘数;
- ④调用 DOS 的 INT 21H 中断的 2 号功能 (MOV AH,2), 显示单个字符 '=';
- ⑤将积转换成 ASCII 字符, 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 显示积;
- ⑥ (此步骤用于判断乘法的结果错误显示) 用 CMP 指令检查错误标志 ERRORFLAG, 若其为 0, 代表结果错误, 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 显示错误提示语句 MSG6。

• 对于除法, 以下是结果显示的步骤:

- ①将被除数转换成 ASCII 字符, 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 显示被除数;
- ②调用 DOS 的 INT 21H 中断的 2 号功能 (MOV AH,2), 显示运算符 '/';
- ③将除数转换成 ASCII 字符, 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 显示除数;
- ④调用 DOS 的 INT 21H 中断的 2 号功能 (MOV AH,2), 显示单个字符 '=';
- ⑤将商转换成 ASCII 字符, 调用 DOS 的 INT 21H 中断的 9 号功能 (MOV AH,9), 显示商;
- ⑥用 CMP 指令比较余数的二进制数是否为 0。若不为 0, 继续以下步骤, 否则结果显示结束;
- ⑦调用子程序 SHENGLUEHAO 显示省略号;
- ⑧将余数转换成 ASCII 字符, (MOV AH,9), 显示余数。

(4) 重复运算算法

该模块位于结果显示模块后。

通过调用 DOS 的 INT 21H 中断的 9 号功能 (MSG7) 询问用户是否继续运算, 通过调用 DOS 的 INT 21H 中断的 1 号功能 (MOV AH,1) 读取用户键入的字符, 若为 'R'/'r', 则继续运算, 否则退出程序。

该模块也可移至输入处理模块, 在判断输入有效性前用 CMP 语句进行判断, 若为 'Q'/'q' 就退出程序, 否则继续进行运算。

(10-15%) 五、程序说明 (含预习报告之流程图修正后的流程图)

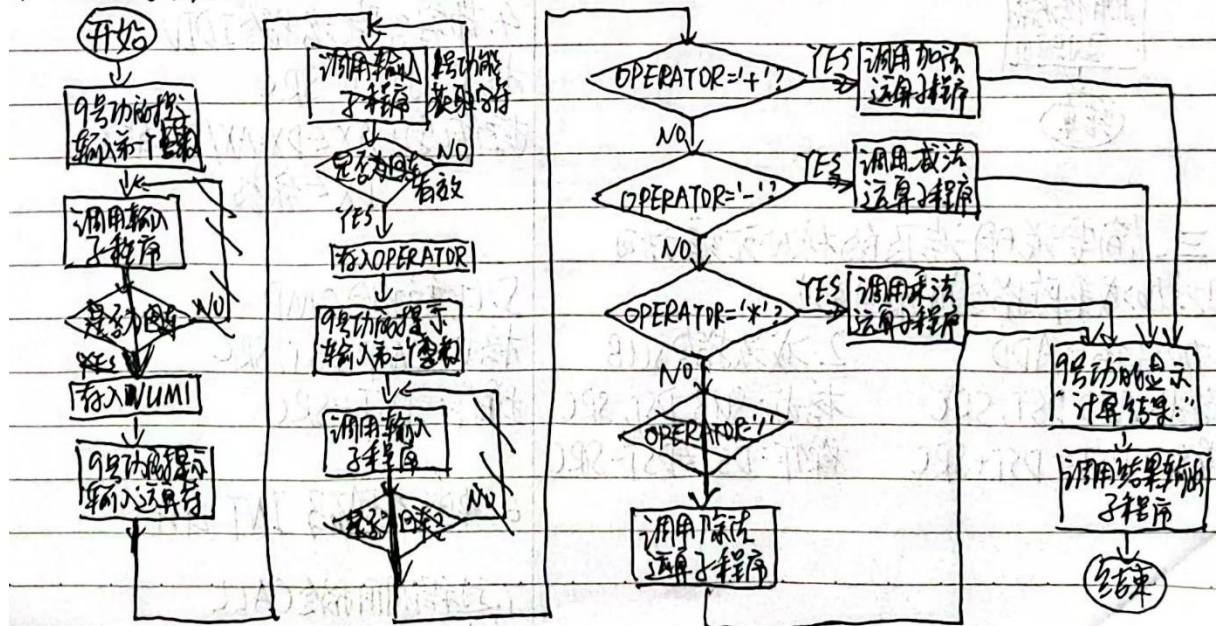
1.程序功能描述:

本程序实现了一个交互式的四位数内正整数四则运算器。它能够接收用户输入的两个 4 位正整数 (范围 0001~9999) 和一个四则运算符 (+、-、*、/), 执行相应的计算并以特定格式输出结果。程序具备输入有效性判断功能, 能对非法数字、非法运算符以及除数为零的情况进行错误提示并要求重新输入。输出格式上, 程序能够自动去除前导零, 加减法采用竖式显示, 乘除法采用横式显示, 除法还能在余数不为零时显示余数。在结果正确性判断方面, 程序能够判断减法结果是否为负、乘法结果是否溢出, 并在出现这些错误时进行提示。运算结束后, 程序会询问用户是否继续, 由此可以实现循环运算, 直到用户选择退出程序。

2. 程序流程图:

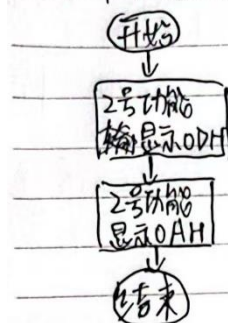
(1) 预习报告的流程图

1. 主程序流程图

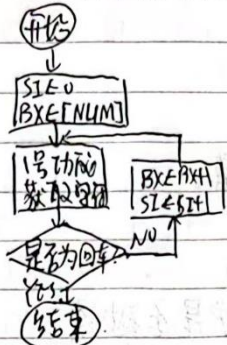


2. 子程序流程图

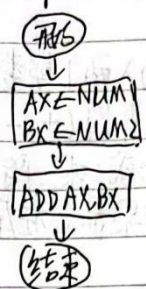
(1) 回车换行子程序



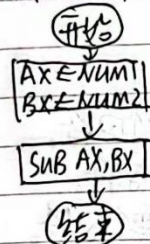
(2) 输入子程序



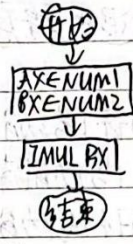
(3) 加法运算子程序



(4) 减法运算子程序



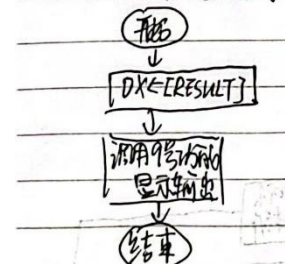
(5) 乘法运算子程序



(6) 除法运算子程序

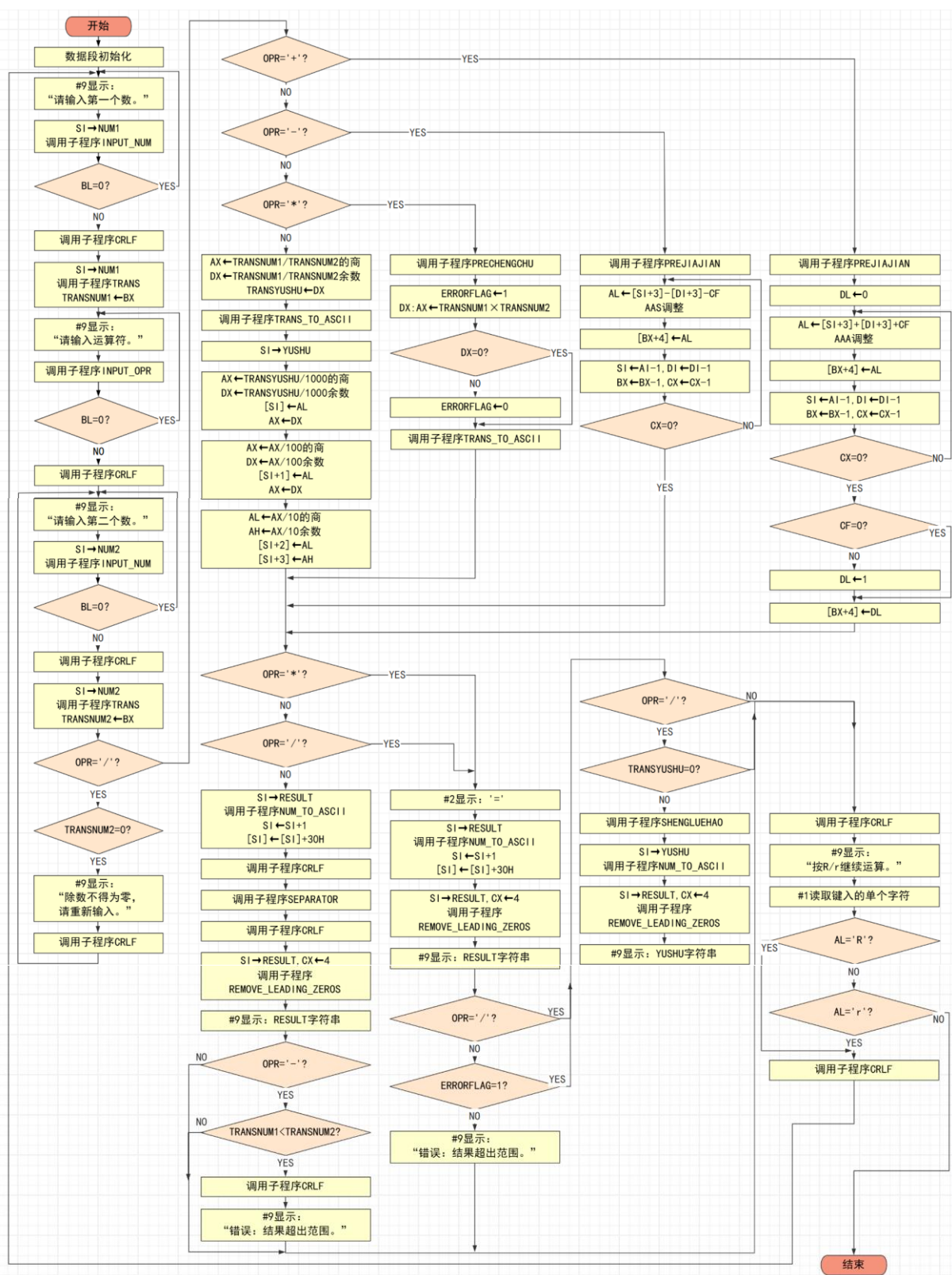


(7) 结果输出子程序

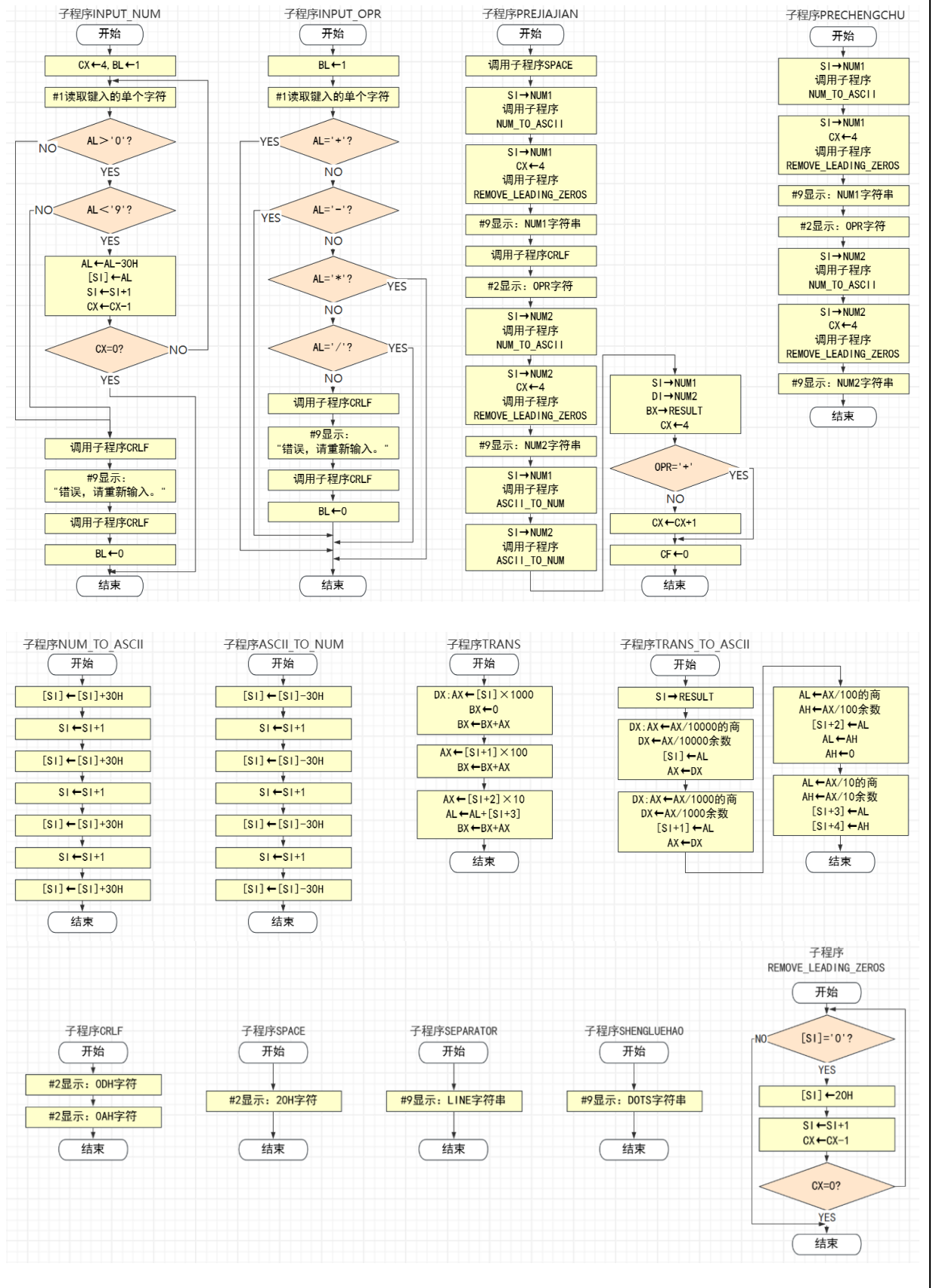


(2) 修正后的流程图

①主程序流程图



②子程序流程图



3.算法与变量、寄存器说明:

(1) 算法: 程序整体上分为输入、计算、输出三个阶段, 并采用主循环结构支持反复运算。

• **输入算法:** 程序通过一个主循环引导用户分步输入第一个数、运算符和第二个数。每个输入环节都调用相应的子程序 (INPUT_NUM, INPUT_OPR), 利用 DOS 中断 INT 21H 的 1 号功能读取键盘输入, 并通过 CMP 指令进行有效性判断。对于数字, 检查是否为 '0'~'9'; 对于运算符, 检查是否为 '+', '-', '*', '/' 中的一个。若输入非法, 则通过标志位 BL 返回错误, 主程序将显示错误信息并重新执行当前输入环节。对于除法, 在输入第二个数后, 会额外检查除数是否为零。

• **计算算法:** 程序首先通过比较运算符 OPR, 采用分支结构跳转至相应的运算处理部分。

加减法: 采用非组合 BCD 码逐位运算算法。通过 PREJIAJIAN 子程序准备数据后, 从个位到最高位循环 4 次, 使用 ADC (带进位加法) 或 SBB (带借位减法) 指令, 并配合 AAA 或 AAS 指令进行十进制调整, 以模拟竖式计算, 同时要进位/借位处理;

乘除法: 采用 16 位二进制数值运算。在运算前, 通过 TRANS 子程序将输入的 BCD 码转换为 16 位二进制数值。乘法使用 MUL 指令, 通过检查结果的高 16 位 (DX 寄存器) 判断是否溢出。除法使用 DIV 指令, 运算后商在 AX 中, 余数在 DX 中, 需分别进行后续处理。

• **输出算法:** 程序根据运算符类型, 将输出流程分为加减法和乘除法两个分支。输出前, 调用 TRANS_TO_ASCII 或 NUM_TO_ASCII 子程序将二进制数值或 BCD 码数值结果转换为 ASCII 字符串, 并调用 REMOVE_LEADING_ZEROS 子程序去除前导零。对于减法和乘法, 会检查错误标志, 如果结果错误, 则显示错误信息 MSG6。对于除法, 会检查余数, 非零则显示省略号和余数。最后, 程序通过 MSG7 提示用户, 并读取按键判断是继续运算 (RESTART) 还是退出 (EXIT)。

(2) 变量说明:

• MSG1~MSG7: 字节型字符串, 用于存放各种提示和错误信息, 以 '\$' 结尾。

• NUM1, NUM2: 字节型数组, ASCII 字符形式或数值形式, 用于存储用户输入的两个数字的非组合型 BCD 码, 供加减法运算和结果显示使用。

• TRANSNUM1, TRANSNUM2: 字型变量, 用于存储将 NUM1 和 NUM2 转换后的完整二进制数值, 供乘除法运算使用。

• OPR: 字节型变量, 用于存储用户输入的运算符字符。

• RESULT: 字节型数组, 用于存储最终计算结果的数值形式, 输出前会转换为 ASCII 字符串。

• YUSHU: 字节型数组, 用于存储余数的数值形式, 输出前会转换为 ASCII 字符串。

• TRANSYUSHU: 字型变量, 用于临时存储除法运算得到的余数二进制数值。

• ERRORFLAG: 字节型变量, 作为错误标志位。1 表示乘法结果正确, 0 表示乘法结果溢出。

• LINE, DOTS: 分隔线、省略号字符串。

(3) 主要寄存器:

• AX: 用于执行无符号数乘法指令 MUL (DX:AX 存储结果)、无符号数除法指令 DIV (AX 作被除数), 以及 DOS 功能调用时的功能号设置 (如 MOV AH,1 读取键入的单个字符并存储在 AL、MOV AH,2 显示字符、MOV AH,9 显示字符串)。

• BX: 在 TRANS 子程序中用于累加最终结果; 在 PREJIAJIAN 子程序中用作指向结果 RESULT 的指针。其低八位 BL 用作子程序 INPUT_NUM 和 INPUT_OPR 的返回标志, 1 表示输入正确, 0 表示输入错误。

• CX: 主要用作 LOOP 指令的循环计数器; 也用作 MUL、DIV 指令中的乘数和除数。

• DX: 在 DIV 指令中存放被除数的高 16 位和运算后的余数, 在 MUL 指令中存放积的高 16 位; 在调用 DOS 的 9 号功能时, 存放待显示字符串的地址。在调用 DOS 的 2 号功能时, 其低八位 DL 用于存放待显示字符的 ASCII 码, DL 也用来存储加法的最高位进位。

• SI, DI: 用作指针, 分别指向 NUM1 和 NUM2, 在加减法循环中逐位访问数据。

• CF: 进位标志位。在加减法运算中, 由 ADC 和 SBB 指令使用, 用于在各位之间传递进位或借位, 是实现多位数运算的关键。

4.核心指令及其用法:

(1) 带进位加法指令 ADC

格式: ADC DST, SRC

操作: $DST \leftarrow DST + SRC + CF$

(2) 带借位减法指令 SBB

格式: SBB DST, SRC

操作: $DST \leftarrow DST - SRC - CF$

(3) 无符号数乘法指令 MUL

格式: MUL SRC

字节操作: $AX \leftarrow AL \times SRC$

字操作: $DX:AX \leftarrow AX \times SRC$

(4) 无符号数除法指令 DIV

格式: DIV SRC

字节操作: $AL \leftarrow AX / SRC$ 的商

$AH \leftarrow AX / SRC$ 的余数

字操作: $AX \leftarrow DX:AX / SRC$ 的商

$DX \leftarrow DX:AX / SRC$ 的余数

(5) 比较指令 CMP

格式: CMP DST, SRC

操作: $DST - SRC$

说明: 该指令执行减法操作, 但不回送结果, 只是根据相减的结果置标志位, 它常用于比较两个数的大小。

(6) 中断类型码 INT 21H

(7) 过程调用指令 CALL

格式: CALL DST

(10-15%) 六、源程序清单及其每句的注释

;定义数据段, 用于存放程序中使用的变量和常量

DATA SEGMENT

MSG1 DB 'Please enter 1st number(0001~9999):','\$';提示输入第一个数字的语句

MSG2 DB 'Please enter the operator(+-*/*):','\$' ;提示输入运算符的语句

MSG3 DB 'Please enter 2nd number(0001~9999):','\$';提示输入第二个数字的语句

MSG4 DB 'Wrong! The divisor cannot be 0. Please re-enter.','\$';除数为 0 的错误提示语句

MSG5 DB 'Wrong! Please re-enter.','\$' ;输入(非数字)错误提示语句

MSG6 DB 'Error: the result is out of range.','\$' ;结果超出范围(如负数)的提示语句

MSG7 DB 'Press R/r to continue.','\$' ;提示是否继续计算的语句

LINE DB '----','\$' ;分隔线字符串

DOTS DB '.....','\$' ;省略号字符串

NUM1 DB 0,0,0,0,'\$' ;存储第一个输入数字(ASCII)

TRANSNUM1 DW 0,'\$' ;转换后的第一个数字(数值形式)

OPR DB 0,'\$' ;存储运算符

```

NUM2 DB 0,0,0,0,'$' ;存储第二个输入数字(ASCII)
TRANSNUM2 DW 0,'$' ;转换后的第二个数字(数值形式)
RESULT DB 0,0,0,0,0,'$';存储计算结果(ASCII)
YUSHU DB 0,0,0,0,'$' ;存储余数(ASCII)
TRANSYUSHU DW 0,'$' ;转换后的余数(数值形式)
ERRORFLAG DB 0,'$' ;错误标志位,用于乘法结果判断
DATA ENDS
;定义代码段,用于存放程序指令
CODE SEGMENT
    ASSUME DS:DATA,CS:CODE ;指定代码段与数据段的关系
START:MOV AX,DATA ;将数据段地址加载到 AX 寄存器
    MOV DS,AX ;设置数据段寄存器 DS 为数据段地址
;-----输入第一个数-----
INPUT1:
    LEA DX,MSG1 ;加载 MSG1 的地址到 DX,准备显示提示信息
    MOV AH,9 ;设置 DOS 功能号 9(显示字符串)
    INT 21H ;调用 DOS 中断 21H,执行显示字符串功能
    LEA SI,NUM1 ;加载 NUM1 缓冲区的地址到 SI 寄存器
    CALL INPUT_NUM ;调用子程序 INPUT_NUM 来读取一个 4 位数
    CMP BL,0 ;检查 INPUT_NUM 子程序返回的标志 BL(0 表示输入错误)
    JE INPUT1 ;如果 BL 为 0(输入错误),则跳回 INPUT1 重新输入
    CALL CRLF ;调用子程序,输出一个回车换行
    LEA SI,NUM1 ;重新加载 NUM1 的地址到 SI
    CALL TRANS ;调用子程序 TRANS,将 ASCII 字符串转换为数值
    MOV TRANSNUM1,BX;将转换后的数值(在 BX 中)存入 TRANSNUM1 变量
;-----输入运算符-----
INPUT2:
    LEA DX,MSG2 ;加载 MSG2 的偏移地址到 DX
    MOV AH,9 ;设置 DOS 功能号 9
    INT 21H ;显示提示输入运算符的信息
    CALL INPUT_OPR ;调用子程序 INPUT_OPR 读取一个运算符
    CMP BL,0 ;检查 INPUT_OPR 返回的标志 BL(0 表示输入错误)
    JE INPUT2 ;如果 BL 为 0(输入错误),则跳回 INPUT2 重新输入
    CALL CRLF ;输出一个回车换行
;-----输入第二个数-----
INPUT3:
    LEA DX,MSG3 ;加载 MSG3 的地址到 DX
    MOV AH,9 ;功能号 9:显示字符串
    INT 21H ;调用 DOS 中断显示提示输入第二个数字的消息
    LEA SI,NUM2 ;加载 NUM2 缓冲区的地址到 SI
    CALL INPUT_NUM ;调用子程序 INPUT_NUM 读取一个 4 位数
    CMP BL,0 ;检查 INPUT_NUM 返回的标志 BL
    JE INPUT3 ;如果输入错误,则跳回 INPUT3 重新输入
    CALL CRLF ;输出一个回车换行
    LEA SI,NUM2 ;重新加载 NUM2 的地址到 SI
    CALL TRANS ;调用子程序 TRANS,将 ASCII 字符串转换为数值
    MOV TRANSNUM2,BX;将转换后的数值(在 BX 中)存入 TRANSNUM2 变量
    CMP OPR, '/' ;检查运算符是否为除法
    JNE CLASSIFIED_DISCUSSION;如果不是除法,跳转到分类讨论
    CMP TRANSNUM2,0 ;如果是除法,检查除数 TRANSNUM2 是否为 0
    JNE CLASSIFIED_DISCUSSION;如果除数不为 0,跳转到分类讨论

```

```

LEA DX,MSG4      ;如果除数为 0,加载错误信息 MSG4 的地址
MOV AH,9         ;设置 DOS 功能号 9
INT 21H          ;显示除数不能为 0 的错误信息
CALL CRLF        ;输出回车换行
JMP INPUT3       ;跳回 INPUT3,要求重新输入第二个数
;-----根据运算符分类进行运算-----
CLASSIFIED_DISCUSSION:
    CMP OPR,'+'   ;比较运算符是否为 '+'
    JE JIAFA      ;如果是,跳转到加法 JIAFA
    CMP OPR,'-'   ;比较运算符是否为 '-'
    JE JIANFA     ;如果是,跳转到减法 JIANFA
    CMP OPR,'*'   ;比较运算符是否为 '*'
    JE CHENGFA    ;如果是,跳转到乘法 CHENGFA
;-----除法运算部分-----
CHUFA:
    CALL PRECHENGCHU ;调用乘除运算前准备子程序
    MOV AX,TRANSNUM1 ;将被除数(第一个数)放入 AX
    MOV CX,TRANSNUM2 ;将除数(第二个数)放入 CX
    XOR DX,DX        ;将 DX 清零,因为字除法 DIV 要求 DX:AX 作被除数
    DIV CX            ;执行无符号除法:AX←DX:AX/CX 的商,DX←余数
    MOV TRANSYUSHU,DX ;将余数(在 DX 中)保存到 TRANSYUSHU 变量
    CALL TRANS_TO_ASCII ;调用子程序,将 AX 中的商转换为 ASCII 字符串存入 RESULT
;将 TRANSYUSHU(ASCII)转为 YUSHU(数值)
    LEA SI,YUSHU      ;加载 YUSHU 的偏移地址到 SI
    MOV AX,TRANSYUSHU ;将余数加载到 AX
    XOR DX,DX         ;清空 DX 寄存器
    MOV CX,1000        ;设置除数 1000,用于提取千位
    DIV CX             ;除以 1000,得到千位
    MOV BYTE PTR[SI],AL ;将千位存入 YUSHU
    MOV AX,DX          ;取余数
    XOR DX,DX         ;清空 DX 寄存器
    MOV CX,100         ;设置除数 100,用于提取百位
    DIV CX             ;除以 100,得到百位
    MOV BYTE PTR[SI+1],AL ;将百位存入 YUSHU
    MOV AX,DX          ;取余数
    MOV CL,10          ;设置除数 10,用于提取十位
    DIV CL             ;除以 10,得到十位
    MOV BYTE PTR[SI+2],AL ;将十位存入 YUSHU
    MOV BYTE PTR[SI+3],AH ;将个位存入 YUSHU
    JMP OUTPUT         ;跳转至输出部分
;-----加法运算部分-----
JIAFA:
    CALL PREJIAJIAN   ;调用子程序,准备加减法的显示格式和数据
    MOV DL,0          ;初始化 DL 为 0,用于存储最终的进位
L1:
    MOV AL,[SI+3]      ;从第一个数的个位开始,取其数值到 AL
    ADC AL,[DI+3]       ;第一、二个数的个位进行带进位相加
    AAA                ;加法的非组合 BCD 数的十进制调整
    MOV [BX+4],AL       ;将加法结果存入结果的个位
    DEC SI              ;SI 指针减 1,指向 NUM1 的高一位
    DEC DI              ;DI 指针减 1,指向 NUM2 的高一位

```

```

    DEC BX                ;BX 指针减 1,指向 RESULT 的高一位
    LOOP L1               ;循环执行 L1,使每一位都相加
    JNC NEXT1             ;如果最高位加法没有产生进位(CF=0),不用进行最高位进位处理
    MOV DL,1              ;如果有进位(CF=1),设置 DL 为 1
NEXT1:
    MOV [BX+4],DL         ;将最终的进位(0 或 1)存入 RESULT 的最高位
    JMP OUTPUT            ;跳转到输出部分
;-----减法运算部分-----
JIANFA:
    CALL PREJIAJIAN       ;调用子程序,准备加减法的显示格式和数据
L2:
    MOV AL,[SI+3]         ;从第一个数的个位开始,取其数值到 AL
    SBB AL,[DI+3]         ;第一、二个数的个位进行带借位相减
    AAS                   ;减法的非组合 BCD 数的十进制调整
    MOV [BX+4],AL         ;将减法结果存入结果的个位
    DEC SI                ;SI 指针减 1,指向 NUM1 的高一位
    DEC DI                ;DI 指针减 1,指向 NUM2 的高一位
    DEC BX                ;BX 指针减 1,指向 RESULT 的高一位
    LOOP L2               ;循环执行 L2,使每一位都相减
    JMP OUTPUT            ;跳转到输出部分
;-----乘法运算部分-----
CHENGFA:
    CALL PRECHENGCHU      ;调用子程序,准备乘除法的显示格式
    MOV AX,TRANSDNUM1     ;将第一个数(被乘数)放入 AX
    MOV CX,TRANSDNUM2     ;将第二个数(乘数)放入 CX
    MUL CX                ;执行无符号乘法:DX:AX←AX×C
    MOV ERRORFLAG,0       ;错误标志位清零
    CMP DX,0              ;检查 DX 是否为 0
    JNE NEXT2             ;如果 DX 不是 0,说明结果溢出
    MOV ERRORFLAG,1       ;如果 DX 是 0,乘法结果正确
NEXT2:
    CALL TRANS_TO_ASCII;调用转换子程序,将结果的数值转换为 ASCII 字符
;-----加减法输出部分-----
OUTPUT:
    CMP OPR,'*'           ;检查运算符是否为乘法
    JE CHENGCHU_OUTPUT    ;如果是乘法,跳转至乘除输出部分
    CMP OPR,'/'           ;判断运算符是否为除法
    JE CHENGCHU_OUTPUT    ;如果是除法,跳转至乘除输出部分
    LEA SI,RESULT         ;加载 RESULT 的偏移地址到 SI
    CALL NUM_TO_ASCII     ;调用数转字符子程序,将数值转换为 ASCII
    INC SI                ;SI 指针加 1,因为该子程序只处理四位
    ADD BYTE PTR[SI],30H;将数值转换为 ASCII 字符
    CALL CRLF             ;输出回车换行
    CALL SEPARATOR        ;调用子程序,输出分隔线'----'
    CALL CRLF             ;输出回车换行
    LEA SI,RESULT         ;重新加载 RESULT 的地址
    MOV CX,4              ;设置循环次数为 4,只需检查加减法结果前 4 位
    CALL REMOVE_LEADING_ZEROS;调用子程序,移除结果字符串前导的'0'
    LEA DX,RESULT         ;加载处理后的 RESULT 地址到 DX
    MOV AH,9              ;设置 DOS 功能号 9
    INT 21H              ;显示加减法最终结果

```

```

CMP OPR,'-'      ;检查运算符是否为减法
JNE AFTER_OUTPUT ;如果不是减法,跳转到 AFTER_OUTPUT
MOV AX,TRANNUM2  ;将第二个数加载到 AX
CMP TRANNUM1,AX  ;比较第一个数和第二个数的大小
JAE AFTER_OUTPUT ;如果第一个数大于等于第二个数,跳转到 AFTER_OUTPUT
CALL CRLF        ;回车换行
LEA DX,MSG6      ;如果结果为负,加载 MSG6 的偏移地址到 DX
MOV AH,9         ;功能号 9:显示字符串
INT 21H          ;调用 DOS 中断显示错误消息
JMP AFTER_OUTPUT ;跳转至输出后处理 AFTER_OUTPUT
;-----乘除法输出部分-----
CHENGCHU_OUTPUT:
MOV DL,'='       ;加载 '=' 字符到 DL
MOV AH,2         ;设置 DOS 功能号 2(显示单个字符)
INT 21H          ;显示 '='
LEA SI,RESULT    ;加载 RESULT 的地址到 SI
CALL NUM_TO_ASCII ;调用数转字符子程序,将数值转换为 ASCII
INC SI           ;SI 指针加 1,因为该子程序只处理四位
ADD BYTE PTR[SI],30H;将数值转换为 ASCII 字符
LEA SI,RESULT    ;重新加载 RESULT 的偏移地址到 SI
MOV CX,4         ;设置去除前导零的次数为 4 次
CALL REMOVE_LEADING_ZEROS;调用去除前导零子程序
LEA DX,RESULT    ;加载 RESULT 的偏移地址到 DX
MOV AH,9         ;功能号 9:显示字符串
INT 21H          ;显示乘法结果或除法的商
CMP OPR, '/'     ;比较运算符是否为 '/'
JE NEXT3         ;如果是,跳转到 NEXT3
CMP ERRORFLAG,1  ;比较错误标志是否为 1
JE NEXT3         ;如果是,结果没有错误,跳转到 NEXT3
CALL CRLF        ;如果不是,输出回车换行
LEA DX,MSG6      ;加载结果超出范围导致错误的语句
MOV AH,9         ;功能号 9:显示字符串
INT 21H          ;显示错误消息
JMP AFTER_OUTPUT ;跳转到 AFTER_OUTPUT
NEXT3:
CMP OPR, '/'     ;比较运算符是否为 '/'
JNE AFTER_OUTPUT ;如果不是,跳转到 AFTER_OUTPUT
CMP TRANSYUSHU,0 ;检查余数是否为 0
JE AFTER_OUTPUT  ;如果余数为 0,跳转到 AFTER_OUTPUT
CALL SHENGLUEHAO ;如果不为 0,调用子程序输出省略号
LEA SI,YUSHU     ;加载余数缓冲区地址
CALL NUM_TO_ASCII ;将余数数值转换为 ASCII
LEA SI,YUSHU     ;重新加载余数缓冲区地址
MOV CX,4         ;设置去除前导零的次数为 4 次
CALL REMOVE_LEADING_ZEROS;调用去除前导零子程序
LEA DX,YUSHU     ;加载处理后的余数地址到 DX
MOV AH,9         ;功能号 9:显示字符串
INT 21H          ;显示余数
;-----输出后处理-----
AFTER_OUTPUT:
CALL CRLF        ;输出回车换行

```

```

    LEA DX,MSG7      ;加载 MSG7 的偏移地址到 DX
    MOV AH,9         ;功能号 9:显示字符串
    INT 21H          ;显示提示信息:是否继续运算?
    MOV AH,1         ;功能号 1:从键盘读取一个字符
    INT 21H          ;读取用户输入
    CMP AL,'R'       ;比较输入是否为'R'
    JE RESTART       ;如果是,跳转到 RESTART
    CMP AL,'r'       ;比较输入是否为'r'
    JE RESTART       ;如果是,跳转到 RESTART
    JMP EXIT         ;否则跳转到 EXIT
RESTART:
    CALL CRLF        ;输出回车换行
    JMP INPUT1       ;跳转到 INPUT1 重新开始
EXIT:
    MOV AH,4CH       ;功能号 4CH:程序结束
    INT 21H          ;调用 DOS 中断结束程序
;子程序 1:输入数
INPUT_NUM PROC NEAR
    MOV CX,4         ;设置输入次数为 4 次
    MOV BL,1         ;初始化标志 BL 为 1(输入正确)
INPUT_LOOP:
    MOV AH,1         ;功能号 1:从键盘读取一个字符
    INT 21H          ;读取用户输入
    CMP AL,'0'       ;比较输入是否小于'0'
    JL WRONG_INPUT_NUM;如果小于,跳转到错误处理
    CMP AL,'9'       ;比较输入是否大于'9'
    JG WRONG_INPUT_NUM;如果大于,跳转到错误处理
    SUB AL,30H        ;将 ASCII 转换为数值
    MOV [SI],AL       ;存储到 NUM1 或 NUM2 缓冲区
    INC SI            ;SI 指向低一位
    LOOP INPUT_LOOP   ;循环四次,完成四位数字的输入
    JMP CORRECT_INPUT ;输入无误,跳转到正确输入处理
WRONG_INPUT_NUM:
    CALL CRLF        ;输出回车换行
    LEA DX,MSG5      ;加载错误信息地址
    MOV AH,9         ;功能号 9:显示字符串
    INT 21H          ;显示错误提示信息
    CALL CRLF        ;输出回车换行
    MOV BL,0         ;设置标志 BL 为 0(输入错误)
CORRECT_INPUT:
    RET              ;子程序返回
INPUT_NUM ENDP
;子程序 2:输入运算符
INPUT_OPR PROC NEAR
    MOV BL,1         ;初始化标志 BL 为 1(输入正确)
    MOV AH,1         ;功能号 1:从键盘读取一个字符
    INT 21H          ;读取用户输入
    MOV OPR,AL       ;存储运算符
    CMP AL,'+'       ;比较是否为 '+'
    JE OPR_CHECKED   ;如果是,跳转到 OPR_CHECKED
    CMP AL,'-'       ;比较是否为 '-'
    JE OPR_CHECKED   ;如果是,跳转到 OPR_CHECKED

```

```

CMP AL,'*'      ;比较是否为'*'
JE OPR_CHECKED ;如果是,跳转到 OPR_CHECKED
CMP AL,'/'      ;比较是否为 '/'
JE OPR_CHECKED ;如果是,跳转到 OPR_CHECKED
WRONG_INPUT_OPR: ;如果不是 '+'-'*'/,
CALL CRLF      ;输出回车换行
LEA DX,MSG5     ;加载错误信息地址
MOV AH,9        ;功能号 9:显示字符串
INT 21H         ;显示错误提示信息
CALL CRLF      ;输出回车换行
MOV BL,0        ;设置标志 BL 为 0(输入错误)
OPR_CHECKED:
RET            ;子程序返回
INPUT_OPR ENDP
;子程序 3:加减法运算前准备
PREJIAJIAN PROC NEAR
CALL SPACE      ;调用子程序输出空格
LEA SI,NUM1     ;加载 NUM1 的偏移地址到 SI
CALL NUM_TO_ASCII ;将 NUM1 中的数值转换为 ASCII
LEA SI,NUM1     ;重新加载 NUM1 的偏移地址到 SI
MOV CX,4        ;设置去除前导零的次数为 4 次
CALL REMOVE_LEADING_ZEROS;调用去除前导零子程序
LEA DX,NUM1     ;加载 NUM1 的偏移地址到 DX
MOV AH,9        ;功能号 9:显示字符串
INT 21H         ;调用 DOS 中断显示第一个数
CALL CRLF      ;回车换行
MOV DL,OPR      ;将运算符加载到 DL
MOV AH,2        ;功能号 2:显示字符
INT 21H         ;调用 DOS 中断显示运算符
LEA SI,NUM2     ;加载 NUM2 的偏移地址到 SI
CALL NUM_TO_ASCII ;将 NUM2 中的数值转换为 ASCII
LEA SI,NUM2     ;重新加载 NUM2 的偏移地址到 SI
MOV CX,4        ;设置去除前导零的次数为 4 次
CALL REMOVE_LEADING_ZEROS;调用去除前导零子程序
LEA DX,NUM2     ;加载 NUM2 的偏移地址到 DX
MOV AH,9        ;功能号 9:显示字符串
INT 21H         ;调用 DOS 中断显示第二个数
LEA SI,NUM1     ;加载 NUM1 的偏移地址到 SI
CALL ASCII_TO_NUM ;将 NUM1 中的 ASCII 转换为数值
LEA SI,NUM2     ;加载 NUM2 的偏移地址到 SI
CALL ASCII_TO_NUM ;将 NUM2 中的 ASCII 转换为数值
LEA SI,NUM1     ;加载 NUM1 的偏移地址到 SI
LEA DI,NUM2     ;加载 NUM2 的偏移地址到 DI
LEA BX,RESULT   ;加载 RESULT 的偏移地址到 BX
MOV CX,4        ;设置循环次数为 4 次
CMP OPR,'+'     ;判断运算符是否为加法
JE NEXT4        ;如果是加法,跳过增加循环次数的操作
INC CX          ;如果是减法,增加一次循环次数(处理借位)
NEXT4:
CLC             ;清除进位标志
RET            ;子程序返回

```

```

PREJIAJIAN ENDP
;子程序 4:乘除法运算前准备
PRECHENGCHU PROC NEAR
    LEA SI,NUM1          ;加载 NUM1 的偏移地址到 SI
    CALL NUM_TO_ASCII ;将 NUM1 中的数值转换为 ASCII
    LEA SI,NUM1          ;重新加载 NUM1 的偏移地址到 SI
    MOV CX,4             ;设置去除前导零的次数为 4 次
    CALL REMOVE_LEADING_ZEROS;调用去除前导零子程序
    LEA DX,NUM1          ;加载 NUM1 的偏移地址到 DX
    MOV AH,9             ;功能号 9:显示字符串
    INT 21H              ;调用 DOS 中断显示第一个数
    MOV DL,OPR           ;将运算符加载到 DL
    MOV AH,2             ;功能号 2:显示字符
    INT 21H              ;调用 DOS 中断显示运算符
    LEA SI,NUM2          ;加载 NUM2 的偏移地址到 SI
    CALL NUM_TO_ASCII ;将 NUM2 中的数值转换为 ASCII
    LEA SI,NUM2          ;重新加载 NUM2 的偏移地址到 SI
    MOV CX,4             ;设置去除前导零的次数为 4 次
    CALL REMOVE_LEADING_ZEROS;调用去除前导零子程序
    LEA DX,NUM2          ;加载 NUM2 的偏移地址到 DX
    MOV AH,9             ;功能号 9:显示字符串
    INT 21H              ;调用 DOS 中断显示第二个数
    RET                  ;返回主程序
PRECHENGCHU ENDP
;子程序 5:数值转 ASCII 字符
NUM_TO_ASCII PROC NEAR
    ADD BYTE PTR[SI],30H;将最高位数值转换为 ASCII
    INC SI                ;指针指向低一位
    ADD BYTE PTR[SI],30H;将当前位转换为 ASCII
    INC SI                ;指针指向低一位
    ADD BYTE PTR[SI],30H;将当前位转换为 ASCII
    INC SI                ;指针指向低一位
    ADD BYTE PTR[SI],30H;将当前位转换为 ASCII
    RET                  ;子程序返回
NUM_TO_ASCII ENDP
;子程序 6:ASCII 字符转数值
ASCII_TO_NUM PROC NEAR
    SUB BYTE PTR[SI],30H;将最高位 ASCII 转换为数值
    INC SI                ;指针指向低一位
    SUB BYTE PTR[SI],30H;将当前位 ASCII 转换为数值
    INC SI                ;指针指向低一位
    SUB BYTE PTR[SI],30H;将当前位 ASCII 转换为数值
    INC SI                ;指针指向低一位
    SUB BYTE PTR[SI],30H;将当前位 ASCII 转换为数值
    RET                  ;子程序返回
ASCII_TO_NUM ENDP
;子程序 7:字符串数值输转整体单个数值
TRANS PROC NEAR
    MOV BX,0              ;初始化 BX 为 0,用于存储转换后的数字
    MOV AL,[SI]           ;将 NUM1 的第一位加载到 AL
    MOV AH,0              ;清空 AH 寄存器
    MOV CX,1000           ;设置乘数为 1000

```

```

MUL CX          ;将 AL 乘以 1000,结果存储在 AX 中
MOV BX,AX       ;将结果存入 BX
MOV AL,[SI+1]   ;将 NUM1 的第二位加载到 AL
MOV AH,0        ;清空 AH 寄存器
MOV CL,100      ;设置乘数为 100
MUL CL          ;将 AL 乘以 100,结果存储在 AX 中
ADD BX,AX       ;将结果加到 BX 中
MOV AL,[SI+2]   ;将 NUM1 的第三位加载到 AL
MOV AH,0        ;清空 AH 寄存器
MOV CL,10       ;设置乘数为 10
MUL CL          ;将 AL 乘以 10,结果存储在 AX 中
ADD AL,[SI+3]   ;将 NUM1 的第四位加到 AL 中
MOV AH,0        ;清空 AH 寄存器
ADD BX,AX       ;将结果加到 BX 中
RET            ;子程序返回

```

TRANS ENDP

;子程序 8:整体单个数值转字符串数值

TRANS_TO_ASCII PROC NEAR

```

LEA SI,RESULT   ;加载 RESULT 的偏移地址到 SI
MOV CX,10000    ;设置除数为 10000
XOR DX,DX       ;清空 DX 寄存器
DIV CX          ;将 AX 除以 10000,商在 AL,余数在 DX
MOV BYTE PTR[SI],AL ;将商(万位)存入 RESULT
MOV AX,DX       ;将余数加载到 AX
XOR DX,DX       ;清空 DX 寄存器
MOV CX,1000     ;设置除数为 1000
DIV CX          ;将 AX 除以 1000,商在 AL,余数在 DX
MOV BYTE PTR[SI+1],AL ;将商(千位)存入 RESULT
MOV AX,DX       ;将余数加载到 AX
XOR DX,DX       ;清空 DX 寄存器
MOV CL,100      ;设置除数为 100
DIV CL          ;将 AX 除以 100,商在 AL,余数在 DX
MOV BYTE PTR[SI+2],AL ;将商(百位)存入 RESULT
MOV AL,AH       ;将余数的高 8 位加载到 AL
XOR AH,AH       ;清空 AH 寄存器
MOV CL,10       ;设置除数为 10
DIV CL          ;将 AL 除以 10,商在 AL,余数在 AH
MOV BYTE PTR[SI+3],AL ;将商(十位)存入 RESULT
MOV BYTE PTR[SI+4],AH ;将余数(个位)存入 RESULT
RET            ;子程序返回

```

TRANS_TO_ASCII ENDP

;子程序 9:回车换行

CRLF PROC NEAR

```

MOV DL,0DH      ;设置回车字符
MOV AH,2        ;功能号 2:显示字符
INT 21H         ;调用 DOS 中断显示回车
MOV DL,0AH      ;设置换行字符
MOV AH,2        ;功能号 2:显示字符
INT 21H         ;调用 DOS 中断显示换行
RET            ;子程序返回

```

CRLF ENDP

```

;子程序 10:空格
SPACE PROC NEAR
    MOV DL,20H           ;设置空格字符
    MOV AH,2             ;功能号 2:显示字符
    INT 21H              ;调用 DOS 中断显示空格
    RET                  ;子程序返回
SPACE ENDP
;子程序 11:线
SEPARATOR PROC NEAR
    LEA DX,LINE           ;加载 LINE 的偏移地址到 DX
    MOV AH,9              ;功能号 9:显示字符串
    INT 21H              ;调用 DOS 中断显示分隔线
    RET                  ;子程序返回
SEPARATOR ENDP
;子程序 12:省略号
SHENGLUEHAO PROC NEAR
    LEA DX,DOTS           ;加载 DOTS 的偏移地址到 DX
    MOV AH,9              ;功能号 9:显示字符串
    INT 21H              ;调用 DOS 中断显示省略号
    RET                  ;子程序返回
SHENGLUEHAO ENDP
;子程序 13:去除前导零
REMOVE_LEADING_ZEROS PROC NEAR
L3:
    CMP BYTE PTR[SI],'0'  ;比较当前字符是否为'0'
    JNE NO_MORE_LEADING_ZEROS;如果不是,跳转到结束
    MOV BYTE PTR[SI],' '  ;将'0'替换为空格
    INC SI                ;指针后移
    LOOP L3               ;循环直到所有前导零被替换
NO_MORE_LEADING_ZEROS:
    RET                  ;子程序返回
REMOVE_LEADING_ZEROS ENDP
CODE ENDS
END START

```

（10-15%）七、上机调试情况说明

1.上机调试步骤

- 无法编译时可根据提示查看源代码出错的地方。
- 编译未出错时查看源程序是否实现功能，未实现说明算法有问题，检查源程序算法。
- 编译成功后，点击“Emulate”进入模拟调试。单步执行，观察每一步各寄存器的值是否符合预期。

2.上机调试过程中遇到的问题是如何解决的

- 用 DL 作错误标志位，但子程序 CRLF 改变了 DL 的值，造成干扰，导致对输入错误的判断错误；改用 BL 作错误标志位，或者也可以新定义一个变量作错误标志位。
- 在二进制数转 BCD 码过程中，除法结果错误；在下一次字除法 DIV 执行前应用 XOR DX,DX 将 DX 清零，因为字除法的被除数是 DX:AX，若上一次除法的余数不为 0，将造成下一次除法的错误。
- CMP TRANSNUM1,TRANSNUM2 报错；两个操作数不能同时为内存单元，可先将字变量 TRANSNUM2 放入 AX，再将字变量 TRANSNUM1 与 AX 进行比较。
- ADD [SI],30H 报错；没有指明目的操作数是字节操作还是字操作，应该改成 ADD BYTE PTR [SI],30H。

3.对调试源程序的中间结果和最终结果进行分析（以 123×456 为例，在 Emu8086 中观察）

A. 中间结果分析：

（1）观察各寄存器的值的变化

- 执行 LEA DX,MSG1 后，DX 为 0000H，因为 MSG1 是第一个定义的，其偏移量为 0。
- 输入 '1'，执行 SUB AL,30H 后，AL 由 31H 变为 01H，说明该条指令将 '1' 的 ASCII 码转换为数值 01H 了，只有转换成数值才能进行后面的竖式计算。
- 随后执行 LOOP INPUT_LOOP，CX 由 0003H 变为 0002H，说明执行完该条指令后还要进行 2 次循环，读取十位和个位数字。观察发现，在输入个位数字 '3' 时，CX 为 0001H，执行 LOOP INPUT_LOOP 后 CX 减一，变为 0000H，由于 CX=0，退出循环，第一个数输入完毕。
- 由于乘法不能列竖式解，只能用 MUL 指令来解，因此需要将 BCD 码转换成二进制数，其中涉及有 MOV CX,1000，MUL CX 等指令。以 MOV CL,100 为例，执行该指令后，CL 变为 64H，64H 化成十进制数正好是 100，符合预期；紧接着执行 MUL CX，AX 由 0001H 变为 0064H，也符合预期，因为字乘法 $AL(01H) \times CX(64H) = AX(0064H)$ 。
- BCD 码转二进制数过程中多次用到 ADD BX,AX 指令，其中 BX 用于存放累加的二进制数，下面就来分析一下 BX 的变化。由于要转换的第一个数是 '0123'，执行第一次 ADD BX,AX 后，其千位为 0，故 BX 为 0000H；执行第二次 ADD BX,AX 后，BX 变为 0064H，即十进制数 100，符合预期；算好十位后结果存在 AX，直接加上个位数字，得到 AX 为 0014H，也即十进制数 23。执行第二次 ADD BX,AX 后，BX 变为 007BH，即十进制数 123，符合预期。
- 执行 LEA DX,MSG2 后，DX 变为 0024H，符合预期，下面就来分析一下。第一个定义的 MSG1 字符串长度为 24H，则存储在 0000H~0023H 共 24 个字节内存单元。MSG 是第二个定义的，因此 MSG2 的偏移地址和偏移量为 0024H，用 LEA 指令将其偏移地址送到 DX 中，使 DX 为 0024H。
- 为了确保乘法结果无误，在执行 MUL CX 前先确定被乘数和乘数正确。执行 MOV AX,TRANSNUM1 后，AX 为 007B，即十进制数 123，符合预期；执行 MOV CX,TRANSNUM2 后，CX 为 01C8，即十进制数 456，符合预期。
- 执行 MUL CX 后，DX 变为 0000H，说明乘法结果没有溢出导致错误；AX 变为 DB18H，即为十进制数 56088，用计算器运算 123×456 得结果为 56088，结果一致，说明乘法结果没有错误。
- 接着需要把乘法结果从二进制数（十六进制数）转换成非组合型 BCD 码以供显示，需要用到多次 DIV CX 或 DIV CL，以 MOV CX,10000，DIV CX 为例。执行 MOV CX,10000 后，CX 变为 2710H，即十进制数 10000；清零 DX 后执行 DIV CX，AX 变为 0005H，DX 变为 17C8，与预期相符。 $56088 \div 10000 = 5 \dots 6088$ ，其中商（5）存到 AX 中，余数（6088，即 17C8H）存到 DX 中。
- 执行 MOV DL,'=' 后，DL 变为 3DH，说明 '=' 的 ASCII 码为 3DH。因此该指令也可以写作 MOV DL,3DH，只是如果不记得其 ASCII 码的话，可以直接写作 '='，注意不能漏掉引号”。
- 在显示结果前，还需要将 BCD 码由数值形式转换成 ASCII 字符串形式，此过程涉及到的寄存器变化不是核心，留待第五步变量变化时观察。
- 结果显示完毕后，输入 'r'，AL 变为 72H，72H 即为 'r' 的 ASCII 码。之后将继续运算。

（2）观察各标志位的变化

- 输入 '1' 后，AL 为 31H，执行 CMP AL,'0'，做减法 $31H - 30H = 01H$ ，不产生借位，则 CF=0，结果 01H 不为 0，则 ZF=0，结果 01H 为正数，则 SF=0；执行 CMP AL,'9'，做减法 $31H - 39H$ ，产生借位，则 CF=1，结果不为 0，则 ZF=0，结果为负数，则 SF=1。
- SUB AL,30H 将字符转换成数值，期间不产生借位，结果为正数，故 CF=0，SF=0。
- 输入 '*' 后，AL 为 2AH，执行 CMP AL,'*'，'*' 的 ASCII 码为 2AH，则做减法 $2AH - 2AH = 00H$ ，不产生借位，则 CF=0，结果 00H 为 0，则 ZF=1，且 SF=0，说明结果 00H 被程序视为正数。随后执行 JE OPR_CHECKED，由于 ZF=1，将产生跳转。

(3) 观察机器码

- LEA DX,MSG1 的机器码是 BA 00 00。BA 是 MOV DX,imm16(16 位立即数)指令的专用操作码,且 LEA DX,MSG1 在逻辑上等同于 MOV DX,OFFSET MSG1,所以该指令的操作码是 BA; 00 00 这两个字节就是 MOV 指令要传送的 16 位立即数,也就是 MSG1 的偏移地址(按小端模式存储)。
- MOV AH,9 的机器码是 B4 09。MOV AH,9 属于 MOV r8(8 位寄存器),imm8(8 位立即数)格式,意思是“将一个 8 位的立即数传送到一个 8 位的寄存器中”。编码规则是:操作码=B0H+寄存器编码。**【8 位寄存器编码为: AL=0,CL=1,DL=2,BL=3,AH=4,CH=5,DH=6,BH=7。】**从中可知 AH 的编码是 4,所以 MOV AH,9 的操作码为 B0H+4H=B4H; 09 是指令要处理的数据,故操作数为 09。
- LEA SI,NUM1 的机器码是 BE F9 00。LEA SI,NUM1 在逻辑上等同于 MOV SI,OFFSET NUM1,属于 MOV r16(16 位寄存器),imm16(16 位立即数)指令格式。编码规则:操作码 = B0H + 寄存器编码。**【16 位寄存器编码为: AX=0,CX=1,DX=2,BX=3,SP=4,BP=5,SI=6,DI=7。】**从中可知,SI 的编码是 6,所以操作码是 B0H+6H = BEH。F9 00 是指令要传送的 16 位立即数,即变量 NUM1 的偏移地址 00F9H,按小端模式存储。
- MOV CX,4 的机器码是 B9 04 00。该指令属于 MOV r16(16 位寄存器),imm16(16 位立即数)格式。查看上述 16 位寄存器编码可知 CX 的寄存器编码是 1,所以操作码是 B0H+1H =B9H。04 00 是 16 位立即数 4 (即 0004H),按小端模式存储。
- MOV BL,1 的机器码是 B3 01。该指令属于 MOV r8(8 位寄存器),imm8(8 位立即数)格式。查看上述 8 位寄存器编码可知 BL 的编码是 3,所以操作码是 B0H+3H=B3H。01 是要传送的 8 位立即数。
- MOV AH,1 的机器码是 B4 01。该指令属于 MOV r8(8 位寄存器),imm8(8 位立即数)格式。查看上述 8 位寄存器编码可知 AH 的寄存器编码是 4,所以操作码是 B0H + 4H = B4H。01 是要传送的 8 位立即数。
- CMP AL,'0'的机器码是 3C 30。3C 是 CMP AL,imm8(8 位立即数)指令的专用操作码。30 是要与 AL 进行比较的 8 位立即数,即字符'0'的 ASCII 码 30H。
- SUB AL,30H 的机器码是 2C 30。2C 是 SUB AL,imm8(8 位立即数)指令的专用操作码。30 是要从 AL 中减去的 8 位立即数 30H。
- MOV [SI],AL 的机器码是 88 04。该指令属于 MOV r/m8,r8 格式,即将寄存器内容传送到内存单元。88 是此指令格式的操作码。04 是 ModR/M 字节,用于指定操作数: Mod=00(表示[SI]寻址,无位移量); Reg=000(表示源寄存器是 AL); R/M=100(表示目的操作数是[SI]),组合成的二进制 00 000 100 即为 04H。
- LOOP INPUT_LOOP 的机器码是 E2 ED。E2 是 LOOP (短循环)指令的固定操作码。ED 是一个 8 位有符号的相对偏移量,其值为 -19 的补码。CPU 执行时会把 CX 减 1,若 CX 不为 0,则将当前 IP 加上这个偏移量,跳转到 INPUT_LOOP 标签处继续循环。
- MOV CX,1000 的机器码是 B9 E8 03。该指令属于 MOV r16(16 位寄存器),imm16(16 位立即数)格式。查看上述 16 位寄存器编码可知 CX 的寄存器编码是 1,所以操作码是 B9H。E8 03 是 16 位立即数 1000 (即 03E8H),按小端模式存储。
- MUL CX 的机器码是 F7 E1。该指令属于 MUL r/m16 格式,执行无符号乘法 AX*CX,结果存于 DX:AX。F7 是一组指令的操作码,具体指令由 ModR/M 字节的 Reg 字段决定。E1 是 ModR/M 字节: Mod=11(表示操作数是寄存器); Reg=100(表示 MUL 指令); R/M=001(表示寄存器是 CX),组合成的二进制 11 100 001 即为 E1H。
- ADD BX,AX 的机器码是 03 D8。该指令属于 ADD r16,r/m16 格式。03 是此指令格式的操作码。D8 是 ModR/M 字节: Mod=11(表示操作数是寄存器); Reg=011(表示目的寄存器是 BX); R/M=000(表示源寄存器是 AX),组合成的二进制 11 011 000 即为 D8H。
- MOV TRANSNUM1,BX 的机器码是 89 1E FE 00。该指令属于 MOV r/m16,r16 格式。89 是此指令格式的操作码。1E 是 ModR/M 字节: Mod=00(表示直接寻址,后面跟 16 位位移量); Reg=011(表示

源寄存器是 BX); R/M=110(表示直接地址模式), 组合成的二进制 00 011 110 即为 1EH。FE 00 是 16 位位移量, 即变量 TRANSNUM1 的偏移地址 00FEH, 按小端模式存储。

- LEA DX,MSG2 的机器码是 BA 24 00。该指令等同于 MOV DX,OFFSET MSG2, 属于 MOV r16(16 位寄存器),imm16(16 位立即数)格式。查看上述 16 位寄存器编码可知 DX 的寄存器编码是 2, 所以操作码是 B0H+2H=BAH。24 00 是变量 MSG2 的偏移地址 0024H, 按小端模式存储。

- CMP AL,'*' 的机器码是 3C 2A。3C 是 CMP AL,imm8(8 位立即数)指令的专用操作码。2A 是要与 AL 进行比较的 8 位立即数, 即字符 '*' 的 ASCII 码 2AH。

- CMPOPR,'/' 的机器码是 80 3E 02 01 2F。该指令属于 CMP r/m8,imm8 格式。80 是此组指令的操作码, 具体指令由 ModR/M 字节的 Reg 字段决定。3E 是 ModR/M 字节: Mod=00(表示直接寻址, 后面跟 16 位位移量); Reg=111(表示 CMP 指令); R/M=110(表示直接地址模式), 组合成的二进制 00 111 110 即为 3EH。02 01 是变量 OPR 的偏移地址 0102H, 按小端模式存储。2F 是要比较的 8 位立即数, 即字符 '/' 的 ASCII 码 2FH。

- ADD BYTE PTR[SI],30H 的机器码是 80 04 30。该指令属于 ADD r/m8,imm8 格式。80 是此组指令的操作码。04 是 ModR/M 字节: Mod=00(表示[SI]寻址, 无位移量); Reg=000(表示 ADD 指令); R/M=100(表示操作数是[SI]), 组合成的二进制 00 000 100 即为 04H。30 是要相加的 8 位立即数 30H。值得一提的是, BYTE PTR 是操作数大小修饰符, 明确指出是字节操作。

- XOR DX,DX 的机器码是 33 D2。该指令属于 XOR r16,r/m16 格式。33 是此指令格式的操作码。D2 是 ModR/M 字节: Mod=11(表示操作数是寄存器); Reg=010(表示目的寄存器是 DX); R/M=010(表示源寄存器也是 DX), 组合成的二进制 11 010 010 即为 D2H。

- DIV CX 的机器码是 F7 F1。该指令属于 DIV r/m16 格式, 执行无符号除法 DX:AX/CX, 商在 AX, 余数在 DX。F7 是此组指令的操作码。F1 是 ModR/M 字节: Mod=11(表示操作数是寄存器); Reg=110(表示 DIV 指令); R/M=001(表示寄存器是 CX), 组合成的二进制 11 110 001 即为 F1H。

(4) 观察反汇编码

- LEA DX,MSG1 的反汇编码是 MOV DX,00000H。因为 LEA DX,MSG1 在逻辑上等同于 MOV DX,OFFSET MSG1, 00000H 即为 MSG1 的偏移地址。同理, 如果 MSG2 的偏移地址为 0024H, 那么 LEA DX,MSG2 的反汇编码就应该是 MOV DX,00024H, 不是小端模式存储。

- LEA SI,NUM1 的反汇编码是 MOV SI,000F9H。同理, LEA SI,NUM1 在逻辑上等同于 MOV SI,OFFSET NUM1, 由此可以读出 NUM1 的偏移地址为 000F9H。

- CALL INPUT_NUM 的反汇编码是 CALL 001E1H, 说明子程序 INPUT_NUM 的入口地址(第一条指令的地址)是 001E1H。

- CMP AL,'0' 的反汇编码是 CMP AL,030H, 可见 '0' 的 ASCII 码为 030H。

- LOOP INPUT_LOOP 的反汇编码是 LOOP 02E6H, 说明标签 INPUT_LOOP 所在的地址是 02E6H。

- MOV AL,[SI+1] 的反汇编码是 MOV AL,[SI]+01H。[SI]是基址寄存器, +01H 是位移量, 其逻辑含义与 MOV AL,[SI+1]完全相同。

- CMPOPR,'+' 的反汇编码是 CMP b.[00102H],02BH。b.前缀指明这是字节操作(因为 OPR 是用 DB 定义的), 00102H 是变量 OPR 的偏移地址, 02BH 是字符 '+' 的 ASCII 码。

- ADD BYTE PTR[SI],30H 的反汇编码是 ADD b.[SI],030H。b.是 BYTE PTR(字节指针)的简写, 两者都用于明确指定操作数是内存中的一个字节, 以消除歧义。

- CMP TRANSYUSHU,0 的反汇编码是 CMP w.[00118H],00H。w.前缀指明这是字(16 位)操作(因为 TRANSYUSHU 是用 DW 定义的), 00118H 是变量 TRANSYUSHU 的偏移地址。

(5) 观察各变量的变化

- 在第一个数的输入过程中, 由于输入的是字符但随即转化为数值, NUM1 的变化过程为:

00H,00H,00H,00H,24H→00H,01H,00H,00H,24H→00H,01H,02H,00H,24H→00H,01H,02H,03H,24H。

- 在子程序 TRANS 中，将 NUM1（非组合型 BCD 码数值形式）转换成 TRANSNUM1（二进制数）。执行此子程序后，TRANSNUM1 由 0 变为 123D。
- 输入运算符后，OPR 由 NULL 变为 '*'。
- 在第二个数的输入过程中，由于输入的是字符但随即转化为数值，NUM2 的变化过程为：00H,00H,00H,00H,24H→00H,04H,00H,00H,24H→00H,04H,05H,00H,24H→00H,04H,05H,06H,24H。
- 在子程序 TRANS 中，将 NUM2（非组合型 BCD 码数值形式）转换成 TRANSNUM2（二进制数）。执行此子程序后，TRANSNUM2 由 0 变为 456D。
- 在子程序 NUM_TO_ASCII 中，将数值转换成字符以供显示，NUM1 的变化过程为：00H,01H,02H,03H,24H→30H,01H,02H,03H,24H→30H,31H,02H,03H,24H→30H,31H,32H,03H,24H→30H,31H,32H,33H,24H。
- 在子程序 REMOVE_LEADING_ZEROS 中，去除 NUM1 的前导零，NUM1 的变化过程为：30H,31H,32H,33H,24H→20H,31H,32H,33H,24H。可见第一位的'0'被空格' '（20H）代替。
- 在子程序 NUM_TO_ASCII 中，将数值转换成字符以供显示，NUM2 的变化过程为：00H,04H,05H,06H,24H→30H,04H,05H,06H,24H→30H,34H,05H,06H,24H→30H,34H,35H,06H,24H→30H,34H,35H,36H,24H。
- 在子程序 REMOVE_LEADING_ZEROS 中，去除 NUM2 的前导零，NUM2 的变化过程为：30H,34H,35H,36H,24H→20H,34H,35H,36H,24H。可见第一位的'0'被空格' '（20H）代替。
- 在乘法运算时，ERRORFLAG 作错误标志位，通过 CMP 语句比较发现 DX 为 0，则乘法结果没有溢出，将 ERRORFLAG 置 1，代表结果正确，不用显示结果错误提示语句。
- 在子程序 TRANS_TO_ASCII 中，将无符号数乘法指令 MUL 得到的积 AX 由二进制数转换成非组合型 BCD 码的数值形式，RESULT 的变化过程为：00H,00H,00H,00H,00H,24H→05H,00H,00H,00H,00H,24H→05H,06H,00H,00H,00H,24H→05H,06H,00H,08H,00H,24H→05H,06H,00H,08H,08H,24H。可见 123×456 的积为 56088。
- 在子程序 NUM_TO_ASCII 中，将数值转换成字符以供显示，RESULT 的变化过程为：05H,06H,00H,08H,08H,24H→35H,06H,00H,08H,08H,24H→35H,36H,00H,08H,08H,24H→35H,36H,30H,08H,08H,24H→35H,36H,30H,38H,08H,24H→35H,36H,30H,38H,38H,24H。

B. 最终结果分析：

程序运行结束后，观察 Emu8086 的 Vars 窗口，RESULT 的内容为 35H,36H,30H,38H,38H,24H，且屏幕上显示 $123 * 456 = 56088$ ，用计算器验证，确为 123 与 456 的积，证明程序运算结果正确。

（10-20%）八、实验结果与分析

1. 分别在 Emu8086（左）和 DICE（右）两个环境下测试不同输入的输出结果，以验证程序运算结果正确性。

分别对加法、减法、乘法、除法进行多次运算，每种运算选取两个示例呈现如下。

（1）加法

$$1234 + 5678 = 6912$$

<pre> Please enter 1st number(0001~9999):1234 Please enter the operator(+-*/*):+ Please enter 2nd number(0001~9999):5678 1234 +5678 ----- 6912 </pre>	<pre> Please enter 1st number(0001~9999):1234 Please enter the operator(+-*/*):+ Please enter 2nd number(0001~9999):5678 1234 +5678 ----- 6912 </pre>
---	---

$$9999+9999=19998$$

Please enter 1st number(0001~9999):9999	Please enter 1st number(0001~9999):9999
Please enter the operator(+-* /):+	Please enter the operator(+-* /):+
Please enter 2nd number(0001~9999):9999	Please enter 2nd number(0001~9999):9999
9999	9999
+9999	+9999
-----	-----
19998	19998

(2) 减法

$$123-45=78$$

Please enter 1st number(0001~9999):0123	Please enter 1st number(0001~9999):0123
Please enter the operator(+-* /):-	Please enter the operator(+-* /):-
Please enter 2nd number(0001~9999):0045	Please enter 2nd number(0001~9999):0045
123	123
- 45	- 45
-----	-----
78	78

$$3456-2789=667$$

Please enter 1st number(0001~9999):3456	Please enter 1st number(0001~9999):3456
Please enter the operator(+-* /):-	Please enter the operator(+-* /):-
Please enter 2nd number(0001~9999):2789	Please enter 2nd number(0001~9999):2789
3456	3456
-2789	-2789
-----	-----
667	667

(3) 乘法

$$123 \times 456 = 56088$$

Please enter 1st number(0001~9999):0123	Please enter 1st number(0001~9999):0123
Please enter the operator(+-* /):*	Please enter the operator(+-* /):*
Please enter 2nd number(0001~9999):0456	Please enter 2nd number(0001~9999):0456
123* 456=56088	123* 456=56088

$$1234 \times 5 = 6170$$

Please enter 1st number(0001~9999):1234	Please enter 1st number(0001~9999):1234
Please enter the operator(+-* /):*	Please enter the operator(+-* /):*
Please enter 2nd number(0001~9999):0005	Please enter 2nd number(0001~9999):0005
1234* 5= 6170	1234* 5= 6170

(4) 除法

$$6000 \div 20 = 300$$

Please enter 1st number(0001~9999):6000	Please enter 1st number(0001~9999):6000
Please enter the operator(+-* /):/	Please enter the operator(+-* /):/
Please enter 2nd number(0001~9999):0020	Please enter 2nd number(0001~9999):0020
6000/ 20= 300	6000/ 20= 300

$$1000 \div 3 = 333 \dots 1$$

Please enter 1st number(0001~9999):1000	Please enter 1st number(0001~9999):1000
Please enter the operator(+-* /):/	Please enter the operator(+-* /):/
Please enter 2nd number(0001~9999):0003	Please enter 2nd number(0001~9999):0003
1000/ 3= 333 \dots 1	1000/ 3= 333 \dots 1

以上四种运算结果与计算器的运算结果一致，从而验证了程序运算结果的正确性。

2. 分别在 Emu8086（左）和 DICE（右）两个环境下测试程序功能，以验证程序功能完整性。

程序的功能主要有人机交互、输入有效性验证、去除前导零、支持反复运算、提示溢出错误，下面将一一验证其功能是否能够正常实现。

（1）人机交互

屏幕上能够显示输入提示语句“请输入第一个数(0001~9999):”、“请输入运算符(+ - × ÷):”、“请输入第二个数(0001~9999):”，程序能够读取键盘键入的数和运算符。

```
Please enter 1st number(0001~9999):0123 Please enter 1st number(0001~9999):0123
Please enter the operator(+-*/*):* Please enter the operator(+-*/*):*
Please enter 2nd number(0001~9999):0456 Please enter 2nd number(0001~9999):0456
```

（2）输入有效性验证

在输入第一个数和第一个数时，若键入的字符不为'0'~'9'，在屏幕上显示错误输入提示语句“错误！请重新输入。”，直至输入的字符全为'0'~'9'才进入下一步骤。

```
Please enter 1st number(0001~9999):, Please enter 2nd number(0001~9999):*
Wrong! Please re-enter. Wrong! Please re-enter.
Please enter 1st number(0001~9999):3A Please enter 2nd number(0001~9999):12r
Wrong! Please re-enter. Wrong! Please re-enter.
Please enter 1st number(0001~9999):0123 Please enter 2nd number(0001~9999):0456
Please enter the operator(+-*/*):_ 123* 456=56088
```

在输入运算符时，若键入的字符不为'+', '-', '*', '/'的其中一个，在屏幕上显示错误输入提示语句“错误！请重新输入。”，直至输入的字符为'+', '-', '*', '/'的其中一个才进入下一步骤。

```
Please enter the operator(+-*/*):` Please enter the operator(+-*/*):?
Wrong! Please re-enter. Wrong! Please re-enter.
Please enter the operator(+-*/*):a Please enter the operator(+-*/*):R
Wrong! Please re-enter. Wrong! Please re-enter.
Please enter the operator(+-*/*):/ Please enter the operator(+-*/*):*
Please enter 2nd number(0001~9999):_ Please enter 2nd number(0001~9999):|
```

运算符为'/'时，输入的第二个数（除数）不得为'0000'，否则在屏幕上显示错误输入提示语句“错误！除数不得为零。请重新输入。”，直至输入非'0000'才进入下一步骤。

```
Please enter the operator(+-*/*):/ Please enter the operator(+-*/*):/
Please enter 2nd number(0001~9999):0000 Please enter 2nd number(0001~9999):0000
Wrong! The divisor cannot be 0. Please re-enter. Wrong! The divisor caQ
Please enter 2nd number(0001~9999):0001 Please enter 2nd number(0001~9999):0001
123/ 1= 123 112/ 1= 112
```

在 DICE 环境下有出现错误输入提示语句显示不正确的情况，但仍有阻止错误输入的功能，以确保结果的正确性。

（3）去除前导零

“去除前导零”的意思是最高位是 0 不显示，除非结果是 0。程序拥有去除前导零的子程序 REMOVE_LEADING_ZEROS，从高位开始，对个位以外的其他位数字进行改'0'为' '的操作，直至该位数字不为'0'时跳出循环。

```
Please enter 1st number(0001~9999):0123 Please enter 1st number(0001~9999):0123
Please enter the operator(+-*/*):+ Please enter the operator(+-*/*):-
Please enter 2nd number(0001~9999):0045 Please enter 2nd number(0001~9999):0045
123 123
+ 45 - 45
-----
168 78
```

<pre> Please enter 1st number(0001~9999):0123 Please enter the operator(+-*/*):- Please enter 2nd number(0001~9999):0123 123 - 123 ----- 0 </pre>	<pre> Please enter 1st number(0001~9999):0456 Please enter the operator(+-*/*):- Please enter 2nd number(0001~9999):0456 456 - 456 ----- 0 </pre>
---	---

(4) 支持反复运算

结果显示后，屏幕上会显示提示语句“输入 R 或 r 继续运算”，此时按下 R 或 r 能够进行下一次运算，否则程序终止。

<pre> Please enter 2nd number(0001~9999):2024 2025 +2024 ----- 4049 Press R/r to continue.r Please enter 1st number(0001~9999):2025 Please enter the operator(+-*/*):- Please enter 2nd number(0001~9999):2024 2025 -2024 ----- 1 Press R/r to continue.r Please enter 1st number(0001~9999):2026 Please enter the operator(+-*/*):* Please enter 2nd number(0001~9999):0025 2026* 25=50650 Press R/r to continue.r Please enter 1st number(0001~9999):2026 Please enter the operator(+-*/*):/ Please enter 2nd number(0001~9999):0025 2026/ 25= 81..... 1 Press R/r to continue.q </pre>	<pre> Please enter 1st number(0001~9999):1114 Please enter the operator(+-*/*):+ Please enter 2nd number(0001~9999):0099 1114 + 99 ----- 1213 Press R/r to continue.r Please enter 1st number(0001~9999):2231 Please enter the operator(+-*/*):- Please enter 2nd number(0001~9999):1221 2231 -1221 ----- 1010 Press R/r to continue.r Please enter 1st number(0001~9999):0099 Please enter the operator(+-*/*):* Please enter 2nd number(0001~9999):0099 99* 99= 9801 Press R/r to continue.r Please enter 1st number(0001~9999):1234 Please enter the operator(+-*/*):/ Please enter 2nd number(0001~9999):5678 1234/5678= 0.....1234 Press R/r to continue.q </pre>
---	--

(5) 提示溢出错误

有两种情况会导致溢出错误，一是差为负数，当输入的第一个数小于第二个数时判断差为负数，结果溢出；二是积大于 65535，执行无符号数字乘法指令后 DX 不为零时判断积大于 65535，结果溢出。下面进行测试：当结果溢出时，屏幕上是否显示溢出错误提示语句“错误：结果超出范围。”

①减法结果溢出

<pre> Please enter 1st number(0001~9999):2025 Please enter the operator(+-*/*):- Please enter 2nd number(0001~9999):2026 2025 -2026 ----- 99999 Error: the result is out of range. </pre>	<pre> Please enter 1st number(0001~9999):1234 Please enter the operator(+-*/*):- Please enter 2nd number(0001~9999):5678 1234 -5678 ----- 95556 Error: the result is out of range. </pre>
---	---

②乘法结果溢出

Please enter 1st number(0001~9999):2024 Please enter the operator(+-*/*):* Please enter 2nd number(0001~9999):2026 2024*2026=37392 Error: the result is out of range.	Please enter 1st number(0001~9999):1221 Please enter the operator(+-*/*):* Please enter 2nd number(0001~9999):0054 1221* 54= 398 Error: the result is out of range.
---	---

测试结果表明，当计算结果发生溢出错误时，程序能够准确识别并显示错误提示，避免用户误将错误结果当作正确值。

通过以上对程序五个主要功能的测试，验证了程序功能的完整性和可靠性。

(15-20%) 九、实验总结(必备部分,评分重要依据之一)

通过本次“基于 PC 的简易整数四则运算器”实验，我将本课程中学到的理论知识与编程实践紧密结合，成功设计并实现了一个功能完善、交互友好的汇编语言应用程序。实验过程不仅使我更加熟悉 8086 汇编语言的指令集、编程方法和调试技巧，还加深了我对微机工作原理、系统交互和程序设计的理解。现结合程序设计与调试过程中的经验、教训和体会，对汇编语言格式、调试方法、汇编指令及汇编语言编程方法、微机原理学习方法以及实验可改进的地方做出总结。

1.汇编语言格式

《微机原理与应用》课程的实验已接近尾声，经过多次实验的程序设计与调试，我基本掌握了汇编语言格式的设计方法。

首先是本次实验相较前几个实验的创新之处，本次实验使用了 13 个子程序，大大减少了代码量，并且增加了可读性。子程序以 PROC NEAR/FAR 开始，以 ENDP 结束，用 CALL 调用，用 RET 返回。通过大量使用子程序，也让我更加熟悉子程序的结构，在编写子程序时更加得心应手。我将知道哪些地方重复度高，从而将这部分代码写进子程序里，进而使程序结构和编程思路更加清晰。

分支结构的核心思想是分类讨论，通过判断运算符 OPR 的类型，使程序走向不同的分支，既能实现多种运算，也能使程序结构和编程思路更加清晰，调试过程更加顺利。

同时，我也总结了一些格式上的易错点。比如错写 ADD [SI],30H，错在没有指明目的操作数是字节操作还是字操作，所以应该改成 ADD BYTE PTR [SI],30H。又如错写 CMP TRANSNUM1,TRANSNUM2，错在目的操作数和源操作数不能同时为内存单元，可先将字变量 TRANSNUM2 放入 AX，再将字变量 TRANSNUM1 与 AX 进行比较。

我还总结了一些比较基础的格式。如：数据段（DATA SEGMENT）与代码段（CODE SEGMENT）的定义和 ENDS 终止，ASSUME 指令关联段寄存器（ASSUME DS:DATA, CS:CODE），数据段的初始化（MOV AX,DATA 和 MOV DS,AX）。

变量需以明确的类型声明（如 DB、DW、DD），字符串需要以 '\$' 结尾，标签后要加上冒号，注释前要有分号。

源操作数和目的操作数的大小需要匹配。需要说明 [SI] 等指针变量的类型（ADD BYTE PTR [SI],30H 明确为字节操作），避免歧义。

无论是已经学过的格式还是新学的格式，只要编写错误就会导致程序的错误，尤其是在要求更严厉的 DICE 环境中，因此编写程序时要格外用心。

2.调试方法

经过多次实验，我已经基本掌握了程序的调试方法，首先将程序调试步骤总结如下：

- （1）先把程序的整体框架写好，在 VARS 窗口查看调试结果是否符合预期。
- （2）如果符合预期，则进行程序功能的完善，在完善过程中观察屏幕和 VARS 窗口，确保功能的正常实现；如果不符合预期，则需要单步调试，比对各寄存器/各变量值与预想值，查看是哪一条指令导致结果出错。

(3) 在结果正确的基础上完善了程序的功能后, 查看是否有报错, 一般是指令的格式错误。如果有报错, 根据提示查看并修改错误指令, 直到程序不报错。

(4) 在程序基本完成, 且确定程序没有错误后, 为每一条指令加上注释。在此过程中, 审视程序的顺序和结构, 删去无必要的指令, 进一步改正细节上的错误和优化程序。

(5) 当程序被确定下来时, 分别观察各寄存器的值的变化、标志位的变化、各变量的变化, 以及机器码和反汇编码。在此过程中, 能够得到更深刻的理解, 实验的收获更大。

具体到本次实验, 我先构建程序的基本架构, 使其能够正确运算, 再完善程序的各种功能。由于本次实验基本框架采用分支结构, 先对加减法部分作调试, 在加减法部分构建并调试好后, 再进行算法与之差异较大的乘除法部分的设计与调试。

输入不同的数和运算符, 尤其是一些特殊值, 就能验证是哪条分支或者哪种情况出错, 由此重点调试到那个部分观察各寄存器和各变量值的变化。找到与预期不符的地方, 就相当于揪住了错处, 对错误指令进行修改后再做进一步调试, 直到多次运算后的结果均正确时, 可以认为该程序是没有问题的。

注释完成后, 观察各寄存器、标志位和变量的变化, 以及机器码和反汇编码。分五遍调试, 每次调试注重观察核心过程中寄存器/标志位/变量的变化, 以及特殊指令的机器码和反汇编码, 并将调试结果记录在实验报告的“上机调试情况说明”部分。

通过来回调试观察, 我发现我触及了汇编语言中更深层次的部分。在观察机器码和反汇编码时, 我能够看懂部分机器码和反汇编码, 是在前几次实验的调试时积累的, 而不懂的部分通过查阅资料, 也让我有了新的理解。同时, 归功于前几次实验积累的调试经验, 本次实验的调试观察过程变得更加游刃有余。

具体到某种调试方法, 我总结了以下几种。可以设置断点来调试程序, 如此一来程序可以停止在断点处, 不会一直运行下去; 可以选择单步运行, 方便一步步地进行调试观察和做记录; 也可以调节“step delay ms”(可调至 200 ms), 使程序以不太快的速度自动运行, 从而更快捷地扫视和检查程序问题, 定位到问题附近时, 按下 STOP 键之后就可以再选择单步运行逐行查找问题。

3. 汇编指令及汇编语言编程方法

根据程序的目标和功能, 合理选择相应的汇编指令。

基本指令, 如传送指令 MOV、LEA 和算术运算指令 ADC、SBB、MUL、DIV, 是程序实现功能的基础。

注意要区分数据传送指令 MOV 和地址传送指令 LEA。例如在 MOV BX,[BX+DI+6H]中, MOV 指令传送的是存储器操作数的内容; 在 LEA BX,[BX+DI+6H]中, LEA 指令传送的是存储器操作数的有效地址(即偏移地址)。

在使用 DOS 的 2 号功能和 9 号功能时也要分清是用 MOV 指令还是 LEA 指令。如果需要显示单个字符, 就要用到 DOS 的 2 号功能, 比如需要显示等于号, 用 MOV 指令(MOV DL, '='); 如果需要显示字符串, 就要用到 DOS 的 9 号功能, 比如需要显示输入提示语句 MSG1, 用 LEA 指令(LEA DX, MSG1)。

算术运算指令 ADC(带进位加法)、SBB(带借位减法)、MUL(无符号数乘法)、DIV(无符号数除法)是实现程序四则运算功能的核心指令。竖式计算需要用到 ADC 指令和 SBB 指令, 需要分别配合调整指令 AAA 和 AAS 使用。使用 MUL 指令和 DIV 指令时, 要分辨出被乘数/被除数、乘数/除数、积/商、余数存放的寄存器, 具体用法可见“程序说明”的“核心指令及其用法”部分。

对于分支结构而言, 其关键指令是 CMP 指令和与之搭配的条件跳转指令, 比如在本实验中就以 CMP 语句判断该进行哪条分支的运算。不仅如此, CMP 指令和与之搭配的条件跳转指令也用于实现输入有效性判断、反复运算等功能。

至于汇编语言编程方法, 最重要的是编程思路。根据程序要实现的功能, 理清编程思路。比如

本次实验的大体编程思路是：先构建好程序的主要框架；在对不同分支进行调试并确定无误后，再增加程序的功能；最后做结构上的优化，并检查程序的细节错误。这种由浅入深的编程方式能够让程序的编写过程事半功倍，如同绘画一般：先铅笔勾勒出轮廓，再用油性笔描边，最后给整幅画涂上颜色；而不是完成画的左半幅后才开始画的右半幅。

4.微机原理学习方法

微机原理的学习，不能只靠理论知识的学习，实践也是重要的一环，而本次实验就是一次生动的实践。

在编写程序时，通过自己动手，无形中熟记了许多核心指令的用法和功能，也有利于对本课程《微机原理与应用》知识点的掌握。在听课时也难免遇到一些难以理解的地方，但是通过认真完成老师布置的作业编程题并进行调试，很多问题都迎刃而解。上次实验中多次运用条件转移指令，因此当我编写分支结构时，我能够更加得心应手地处理这些条件转移指令，并将其在该程序中多处运用，更加加深了我对这些指令的用法、功能和原理的理解；在期中考试，我能判断出 `MOV [BX],20` 指令是错误的并说明错误原因，该指令的错误原因是没有指明目的操作数是字节操作还是字操作。但是在编写程序时又犯了相同的错误，我将 `ADD BYTE PTR [SI],30H` 错写成 `ADD [SI],30H` 了。即便又犯这种经典错误，通过实践，我能够对这种错误有更深刻的印象，以后无论是答题还是编写程序时也能够更少犯这种错误。

编写程序只是实验中的一部分，其后的调试观察过程更为重要，十分有利于《微机原理与应用》的学习，尤其是对机器码和反汇编码的观察。对机器码进行观察，我知悉了操作码的编码规则和各寄存器的编号，触及了微机原理中更深的层次。对反汇编码进行观察，我发现了 `LEADX,MSG1` 与 `MOV DX,OFFSET MSG1` 在逻辑上是等同的，观察调用子程序指令 `CALL` 的反汇编代码，我能直接读出子程序入口的偏移地址。观察寄存器、标志位和变量的变化，让我掌握了程序运行的逻辑，以及各指令的用法、功能、原理及其之间相互的关系。

本次实验让我认识到，学习微机原理必须理论与实践相结合，自底向上地构建知识体系。仅仅背诵指令的功能是远远不够的，必须通过编写程序，去理解 CPU 如何取指、译码、执行；如何通过寄存器暂存数据；如何通过地址总线访问内存。调试过程本身就是一个绝佳的学习机会，当程序出错时，我被迫去思考：AAA 指令和 AAS 指令是如何工作的？为什么执行字除法 DIV 指令前需要清零 DX？为什么结果显示会出现乱符？对这些“为什么”的探寻，让我对微机内部工作原理的理解从抽象概念变为了具体、生动的知识。

5.实验程序改进

由于实验时间有限，未来可做以下改进：

- (1) 增强程序功能，如：增强交互友好程度（优化屏幕显示、支持动态输入、支持用户主动退出）、扩展运算范围（负数、多位数、浮点数）、增加运算方式（混合运算、乘方、开方）等。
- (2) 优化主程序和子程序结构，使程序结构更清晰可辨。
- (3) 处理细节问题，如编写更加清晰易懂的标签和子程序名称。

(10-15%) 十、思考题：

无。

指导教师批阅意见：			
成绩评定：			
预习情况（20）	实验过程（35）	实验报告（45）	总分
指导教师签字： 年 月 日			
备注：			

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
- 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。
- 3、实验过程与实验报告评分比例会根据具体实验来调整。
- 4、务必把实验课前老师签字了的预习报告附上，否则不予评分。